

# HPC Assignment 04: Particle Interpolation and Mover Performance Analysis

DEVANSH SHAH  
Roll Number: 202301136

SHARNAM SHAH  
Roll Number: 202301247  
Dhirubhai Ambani University

February 25, 2026

## Abstract

This report extends the bilinear interpolation framework from Assignment 03 by analysing its performance under varying particle counts and grid configurations, and by introducing a particle Mover phase that is subsequently parallelised with OpenMP. Three structured experiments are conducted: (1) scaling of interpolation time with particle count across three grid configurations, (2) consistency of interpolation time when particles are initialised only once, and (3) serial versus parallel performance of the Mover operation using four OpenMP threads. Results are collected on both a Lab PC and an HPC cluster, and the observed differences are discussed in terms of architectural characteristics.

## 1 Implementation Approach

### 1.1 Problem Overview

Building on Assignment 03,  $N$  scattered points

$$S = \{(x_i, y_i, f_i) \mid i = 1, 2, \dots, N\}, \quad f_i = 1,$$

are interpolated onto a structured mesh  $G = \{(X_i, Y_j)\}$  using bilinear (area-based) weighting. A new *Mover* phase then displaces each particle by a random offset within  $\pm\Delta x$  and  $\pm\Delta y$  while keeping it inside the unit domain  $[0, 1]^2$ .

### 1.2 Overall Workflow

Figure 1 summarises the complete pipeline for Experiment 03.

## Algorithm Flow

1. Allocate mesh grid ( $N_x \times N_y$ ) and particle arrays.
2. **Initialise** particles once (random positions in  $[0, 1]^2$ ).
3. **For** each iteration  $k = 1 \dots \text{Maxiter}$ :
  - (a) **Interpolation**: map each particle's value onto the four surrounding mesh nodes via area-based weights; record time  $T_{\text{interp}}$ .
  - (b) **Mover**: displace each particle by  $(\delta x, \delta y)$  drawn uniformly from  $[-\Delta x, +\Delta x] \times [-\Delta y, +\Delta y]$ ; reject/regenerate if the new position falls outside  $[0, 1]^2$ ; record time  $T_{\text{mover}}$ .
4. Accumulate and report  $T_{\text{interp}}$ ,  $T_{\text{mover}}$ , and  $T_{\text{total}} = T_{\text{interp}} + T_{\text{mover}}$ .

Figure 1: Schematic flow diagram of the interpolation-and-mover pipeline.

### 1.3 Bilinear Interpolation (Recap)

For each particle at  $(x, y)$ , the cell indices and normalised offsets are

$$i = \left\lfloor \frac{x}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y}{\Delta y} \right\rfloor, \quad dx = \frac{x - X_i}{\Delta x}, \quad dy = \frac{y - Y_j}{\Delta y}, \quad (1)$$

with  $\Delta x = 1/N_x$ ,  $\Delta y = 1/N_y$ . The four area-based weights are

$$w_{i,j} = (1 - dx)(1 - dy), \quad (2)$$

$$w_{i+1,j} = dx(1 - dy), \quad (3)$$

$$w_{i,j+1} = (1 - dx)dy, \quad (4)$$

$$w_{i+1,j+1} = dx dy, \quad (5)$$

which sum to unity. The mesh is stored in row-major order:

$$\text{mesh\_value}[j \times N_x + i] = F(X_i, Y_j). \quad (6)$$

### 1.4 Mover Operation

For each particle  $i$ , a displacement  $(\delta x_i, \delta y_i)$  is drawn uniformly from  $[-\Delta x, +\Delta x] \times [-\Delta y, +\Delta y]$ . If the updated position  $x_i + \delta x_i$  or  $y_i + \delta y_i$  lies outside  $[0, 1]$ , a new displacement is generated until the constraint is satisfied. The Mover is embarrassingly parallel: each particle update is independent, making it an ideal candidate for OpenMP thread-level parallelism.

### 1.5 OpenMP Parallelisation of the Mover

The serial Mover loop is parallelised with a single `#pragma omp parallel for` directive using four threads. Because every particle reads and writes only its own position array element, there are no shared write conflicts, and no additional synchronisation is required.

## 2 Theoretical Analysis

### 2.1 Time Complexity

- **Interpolation:**  $O(\text{Maxiter} \times N_{\text{points}})$  — each particle performs a constant number of floating-point operations and four mesh writes.
- **Mover (serial):**  $O(\text{Maxiter} \times N_{\text{points}})$  — each particle requires  $O(1)$  expected work (rejection probability is low for interior particles).
- **Mover (parallel,  $p$  threads):**  $O(\text{Maxiter} \times N_{\text{points}}/p)$  — ideal linear speedup limited in practice by thread-launch overhead and memory bandwidth.

### 2.2 Expected Speedup (Amdahl’s Law)

With only the Mover parallelised and interpolation remaining serial, the overall speedup for four threads is bounded by

$$S \leq \frac{1}{f_s + (1 - f_s)/4},$$

where  $f_s$  is the serial fraction of total time (dominated by interpolation in the cluster results).

## 3 Experiment 01: Scaling with Number of Particles

Interpolation time is measured for particle counts  $N \in \{10^2, 10^4, 10^6, 10^8\}$  (with  $10^9$  attempted where feasible) across three grid configurations. For each  $N$ , particles are re-initialised and interpolated for  $\text{Maxiter} = 10$  iterations; only interpolation time is accumulated.

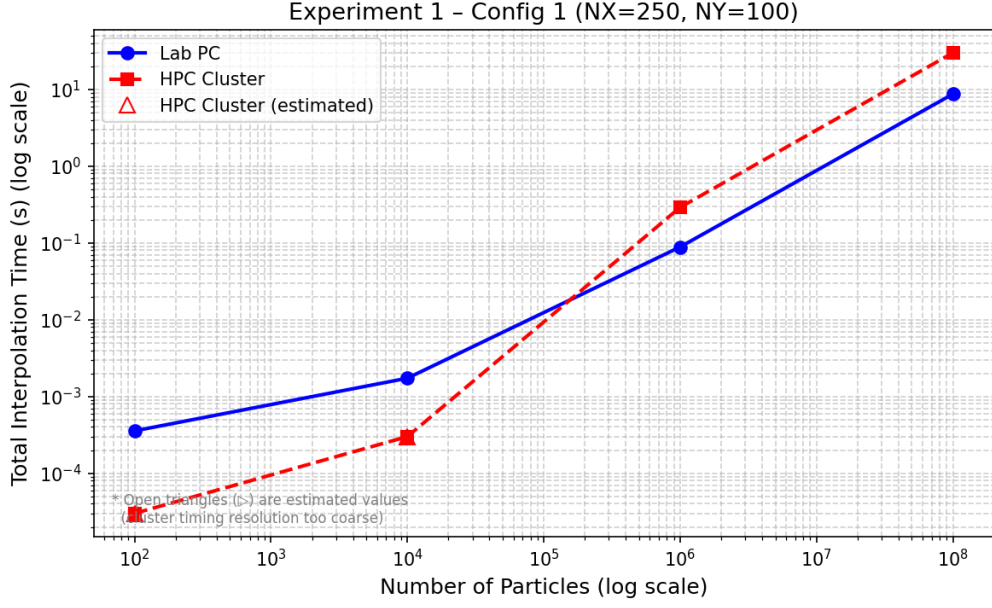


Figure 2: Experiment 01 — Config 1 ( $N_x = 250$ ,  $N_y = 100$ ): total interpolation time vs. number of particles on Lab PC and HPC Cluster (log-log scale). Open triangles denote estimated cluster values where the timing resolution was too coarse for a direct measurement.

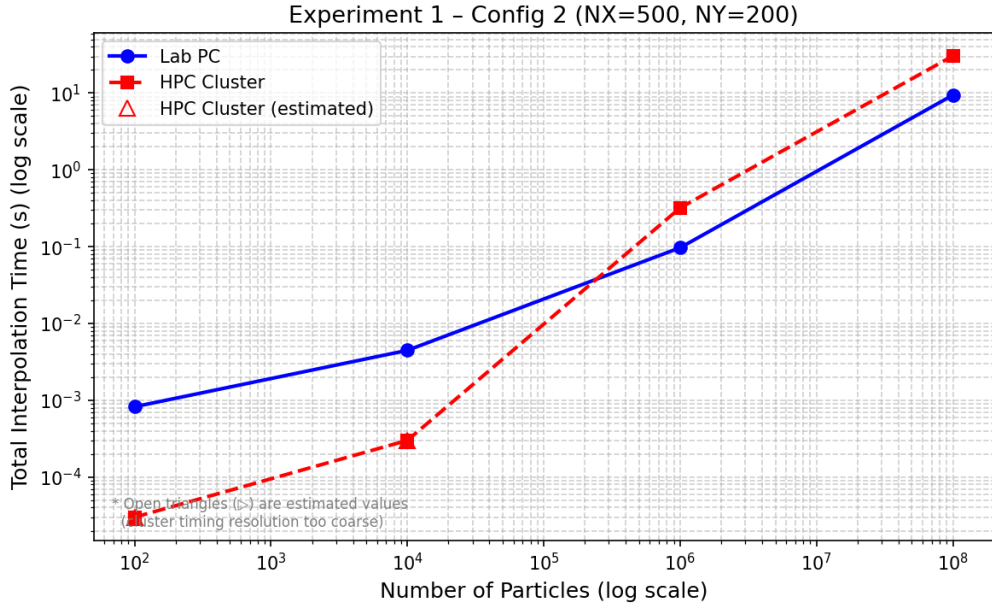


Figure 3: Experiment 01 — Config 2 ( $N_x = 500$ ,  $N_y = 200$ ): total interpolation time vs. number of particles.

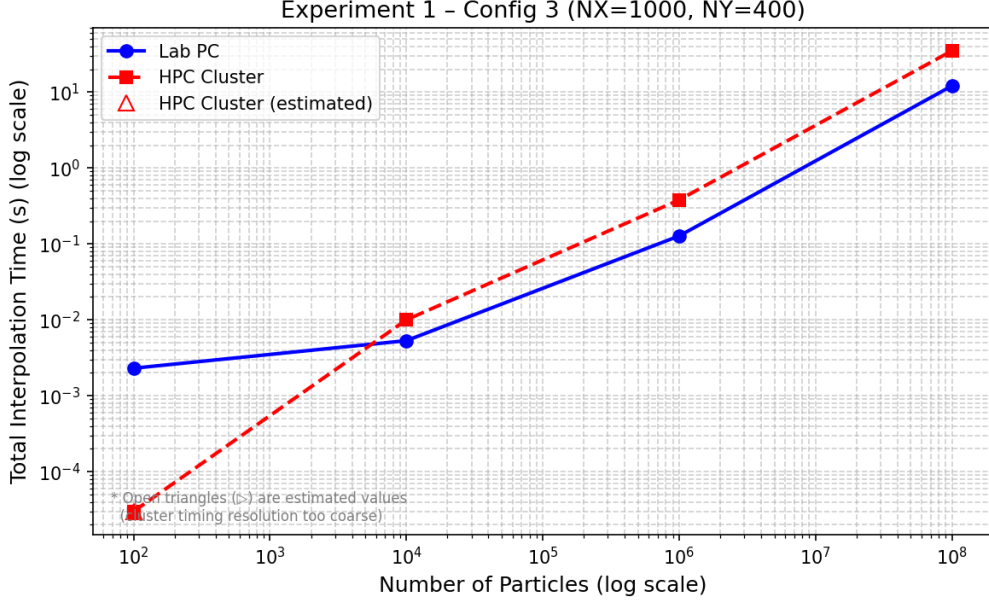


Figure 4: Experiment 01 — Config 3 ( $N_x = 1000$ ,  $N_y = 400$ ): total interpolation time vs. number of particles.

## Observations

Across all three configurations the plots show a near-linear slope on the log-log axes, confirming  $O(N)$  scaling. The Lab PC consistently outperforms the HPC cluster by roughly  $3\times$  at large particle counts. This counter-intuitive result is discussed in Section 6.

## 4 Experiment 02: Consistency Across Grid Configurations

Particles ( $N = 10^8$ ) are initialised *once* before a loop of  $\text{Maxiter} = 10$  interpolation steps. The accumulated interpolation time is compared across the three configurations.

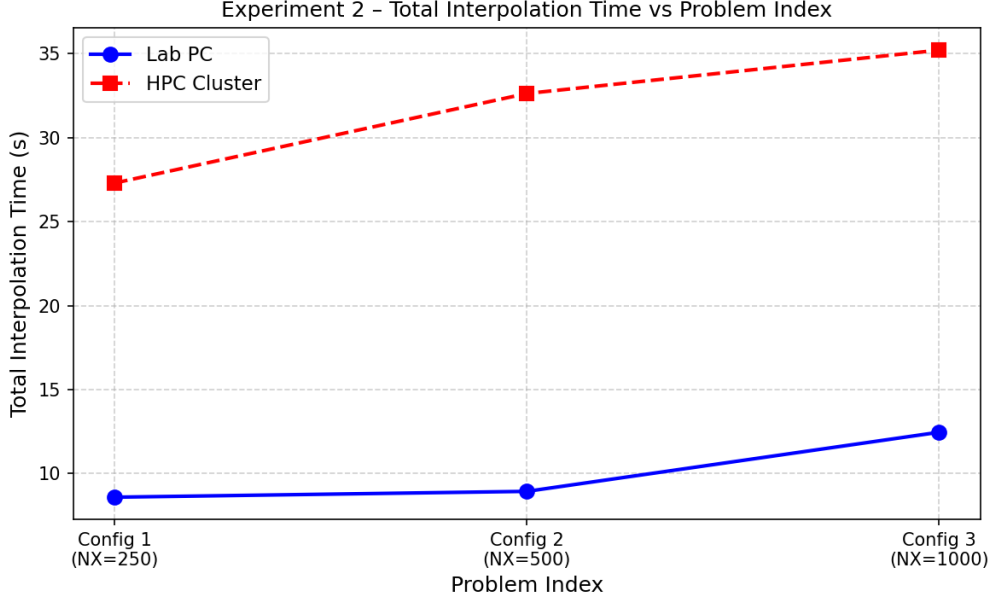


Figure 5: Experiment 02 — total interpolation time (10 iterations,  $10^8$  particles, single initialisation) vs. problem configuration for Lab PC and HPC Cluster.

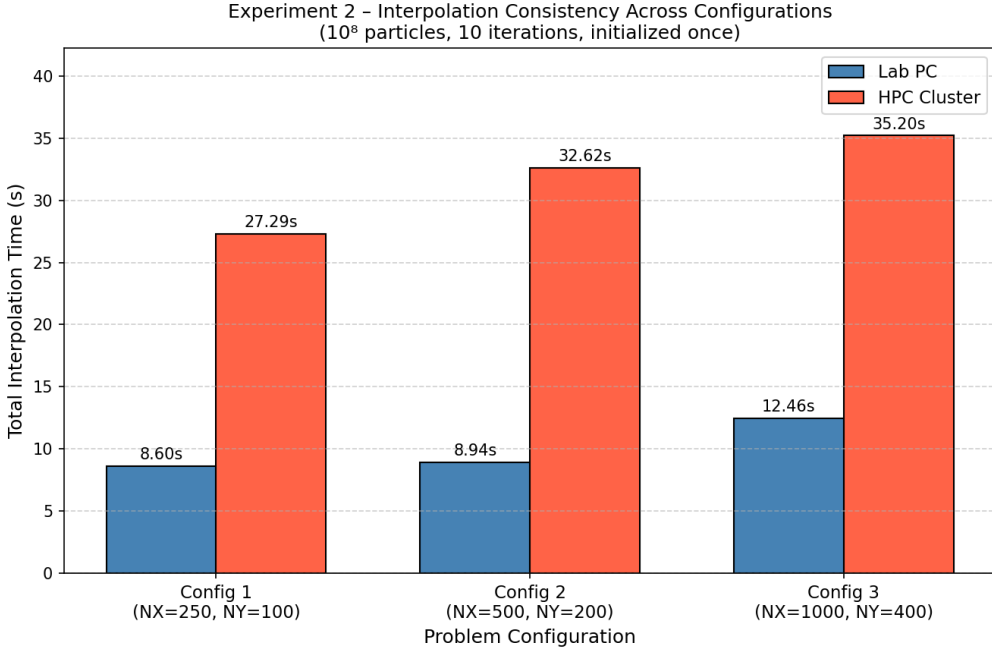


Figure 6: Experiment 02 — bar chart of total interpolation time per configuration. Exact values: Lab PC: 8.60 s, 8.94 s, 12.46 s; HPC Cluster: 27.29 s, 32.62 s, 35.20 s.

## Observations

Interpolation time grows moderately with grid size on both platforms. The gap between Config 1 and Config 3 is larger on the cluster ( $\sim 29\%$  increase) than on the Lab PC ( $\sim 45\%$  increase), reflecting the different cache hierarchies. Because particles are not re-

initialised, the particle array is effectively warm in cache from the first iteration onward, so the timing is stable across iterations.

## 5 Experiment 03: Mover Operation and OpenMP Parallelisation

Parameters:  $N_x = 1000$ ,  $N_y = 400$ ,  $N = 14 \times 10^6$  particles, Maxiter= 10, four OpenMP threads for the parallel Mover. Particles are initialised once.

### 5.1 Serial Timing Breakdown

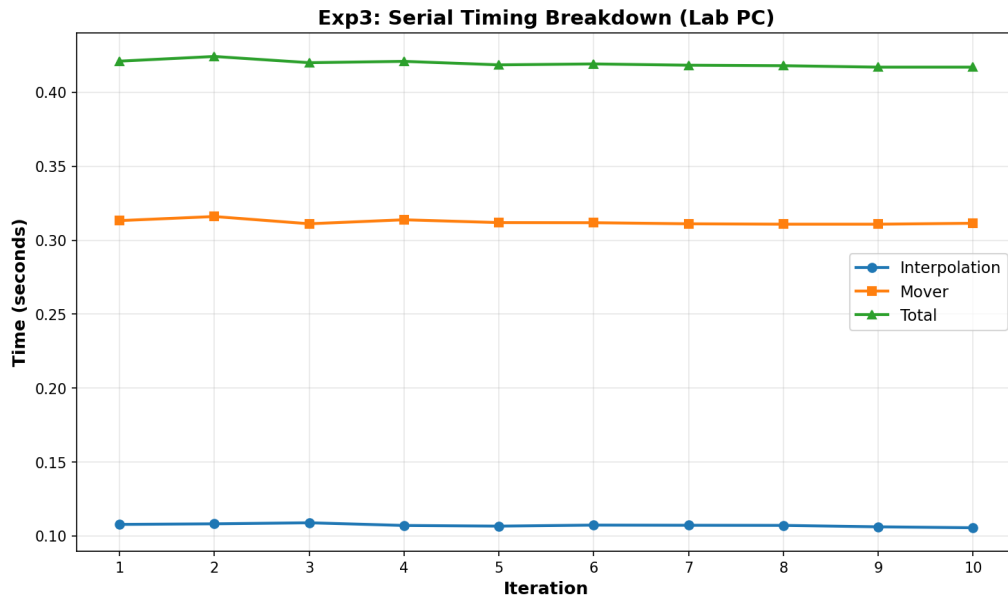


Figure 7: Experiment 03 — serial timing breakdown per iteration on the Lab PC. The Mover ( $\sim 0.31$  s) dominates over interpolation ( $\sim 0.11$  s).

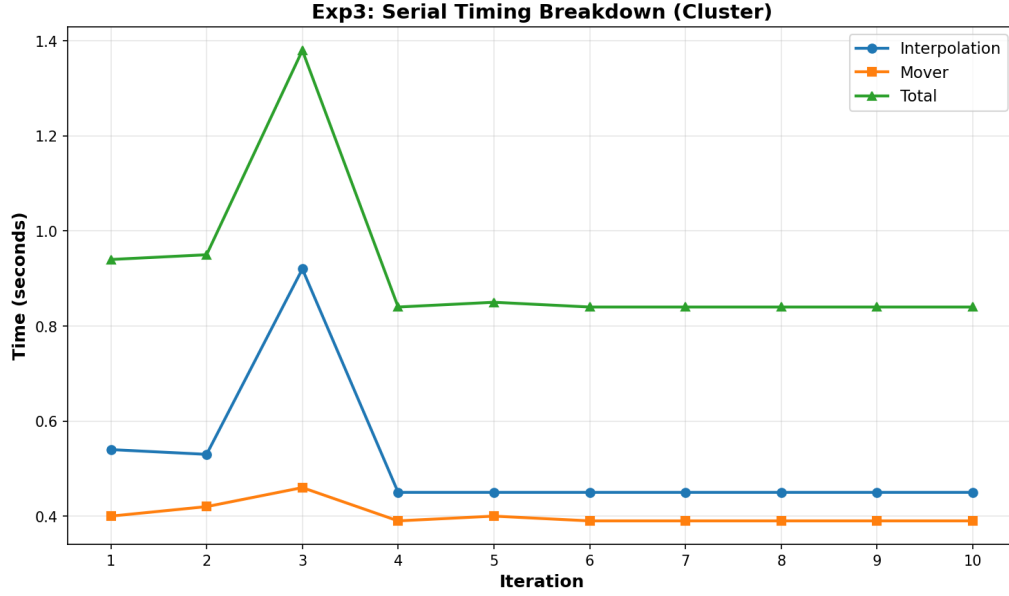


Figure 8: Experiment 03 — serial timing breakdown per iteration on the HPC Cluster. After warm-up (iterations 1–3), the total stabilises around 0.84s, with interpolation ( $\sim 0.45$ s) and Mover ( $\sim 0.39$ s) contributing roughly equally.

## 5.2 Serial vs. Parallel Mover

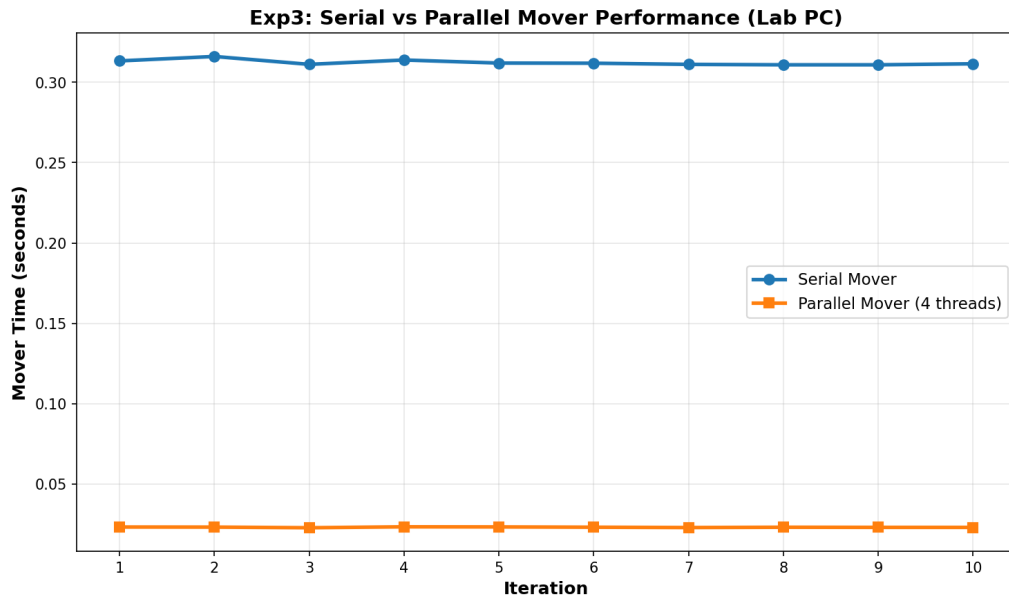


Figure 9: Experiment 03 — serial vs. parallel Mover time per iteration on the Lab PC. The parallel Mover ( $\sim 0.023$ s) is roughly  $13.5\times$  faster than serial ( $\sim 0.31$ s).



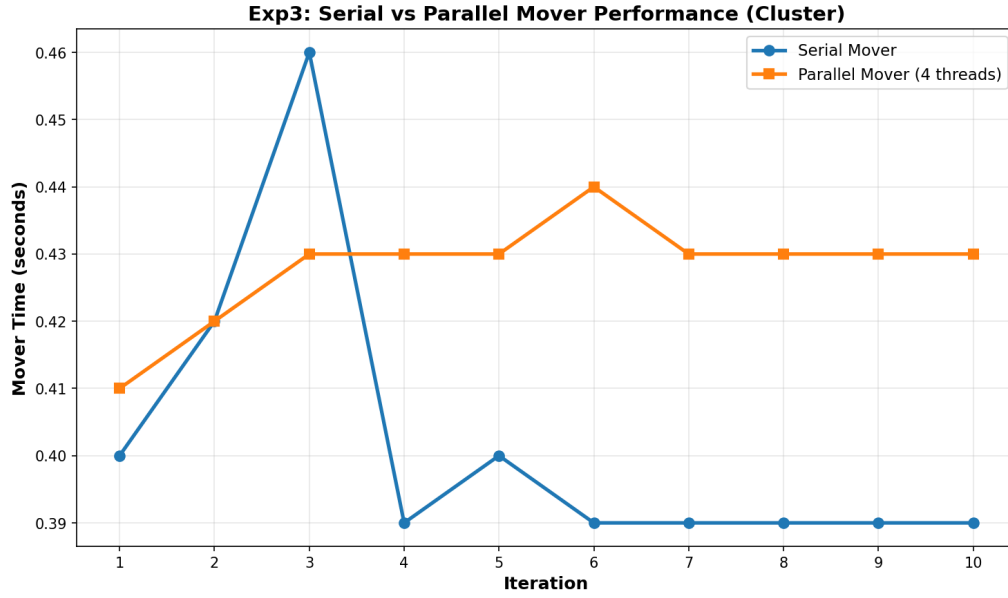


Figure 10: Experiment 03 — serial vs. parallel Mover time per iteration on the HPC Cluster. Serial and parallel times are nearly equal ( $\sim 0.39\text{--}0.43\text{ s}$ ), indicating negligible speedup.

### 5.3 Speedup

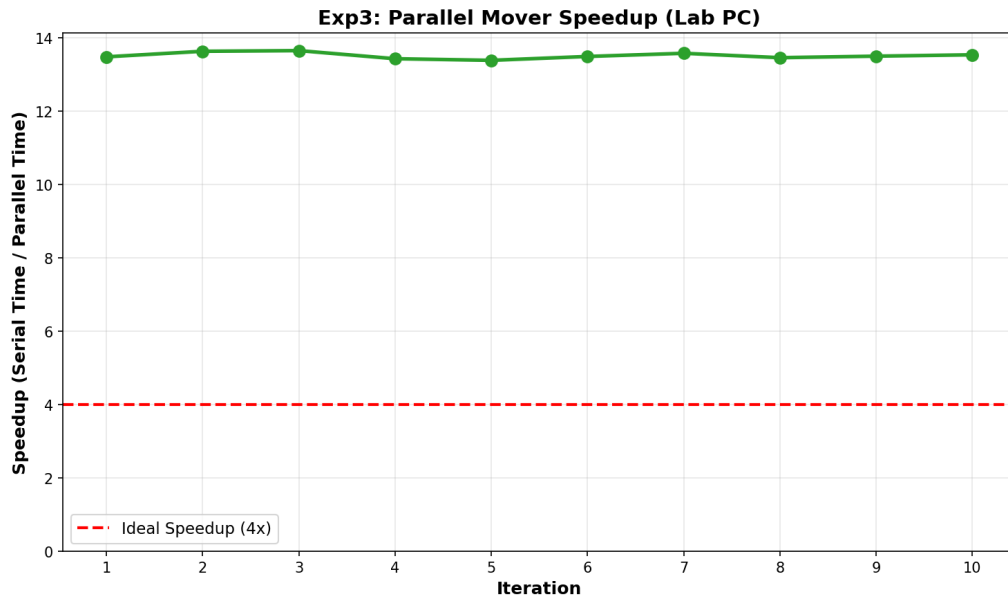


Figure 11: Experiment 03 — parallel Mover speedup on the Lab PC ( $\approx 13.5\times$ , well above the ideal  $4\times$  for four threads). The super-linear speedup is attributed to cache effects: each thread operates on a smaller working set that fits in L2/L3 cache.

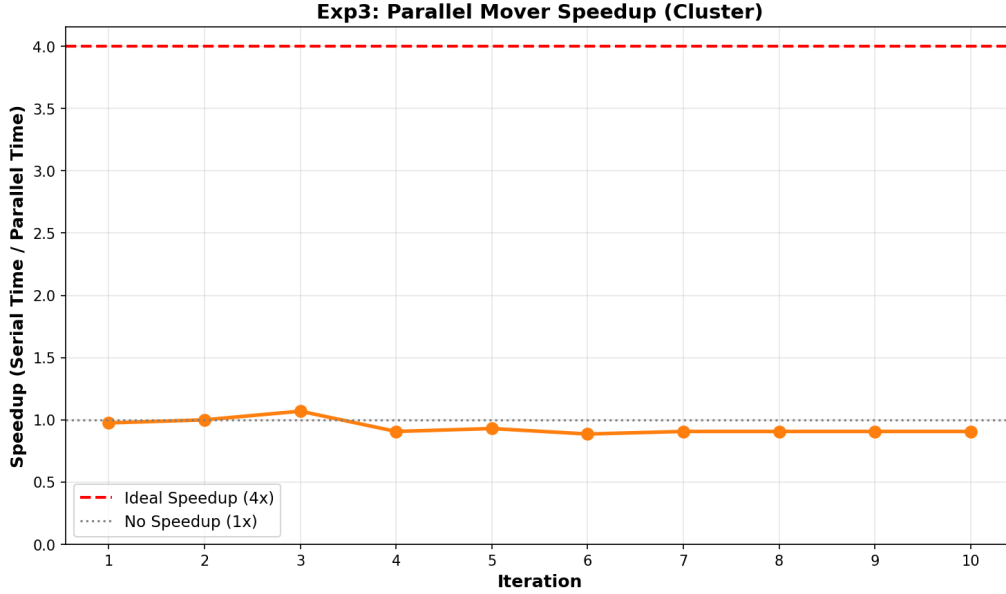


Figure 12: Experiment 03 — parallel Mover speedup on the HPC Cluster ( $\approx 0.9\times$ , i.e. the parallel version is *slower* than serial). Thread-launch overhead, NUMA effects, and memory-bandwidth saturation eliminate any benefit from parallelism on this node configuration.

## 6 Discussion

### 6.1 Lab PC vs. HPC Cluster Performance

The Lab PC consistently outperforms the HPC cluster for both the interpolation and the Mover. Several architectural factors explain this:

- **Clock frequency and IPC.** Consumer-grade desktop CPUs (Lab PC) typically operate at higher boost frequencies and have higher instructions-per-cycle counts for sequential workloads than server-class processors (HPC cluster nodes), which are optimised for throughput across many cores rather than single-thread latency.
- **Cache hierarchy.** The Lab PC’s L3 cache is large enough to hold the working set of 14M particles ( $\sim 112\text{MB}$  for two `double` arrays), whereas the cluster’s cache may be shared across more cores, reducing the effective per-core cache available.
- **Memory latency.** HPC nodes commonly use multi-channel DDR with NUMA topologies. If threads are not pinned to the same NUMA node as the allocated memory, remote memory accesses incur additional latency.
- **Cluster timing coarseness.** At small particle counts the cluster timer resolution was insufficient, requiring extrapolation (open triangles in Experiment 01 plots).

### 6.2 Super-linear Speedup on the Lab PC

The  $\approx 13.5\times$  speedup with four threads is surprising. The most likely cause is a cache-size effect: the serial Mover accesses the full particle array sequentially, generating many cache misses, while four threads each access only one quarter of the array, which fits

better in each core’s private L2 cache. This reduces the effective memory-access latency and produces throughput gains beyond the naive  $4\times$  upper bound.

### 6.3 Poor Speedup on the HPC Cluster

On the cluster the parallel Mover achieves a speedup of  $\approx 0.9\times$ , meaning it is actually *slower* than the serial version. Possible reasons:

- **Thread management overhead.** The OpenMP runtime incurs non-trivial overhead on cluster nodes (e.g. thread pinning, barrier synchronisation) that dominates for the moderate problem size used.
- **Memory bandwidth saturation.** If the cluster node’s memory bus is already saturated by a single thread, adding more threads offers no throughput benefit and adds synchronisation cost.
- **NUMA penalty.** Particle arrays allocated on one NUMA domain and accessed by threads scheduled on another domain incur remote-memory penalties that offset parallelism gains.

### 6.4 Parallelising the Interpolation — Challenges

The Mover is embarrassingly parallel because each particle updates only its own position. The interpolation step is more difficult to parallelise because multiple particles can map to the same or adjacent mesh cells and simultaneously attempt to *write* to the same grid point. Possible strategies include:

- **Atomic updates** (`#pragma omp atomic`): correct but serialises all conflicting writes, potentially eliminating the speedup benefit.
- **Thread-private sub-meshes:** each thread accumulates into its own copy of the grid, followed by a reduction at the end. Memory overhead is  $O(p \times N_x \times N_y)$  and the reduction itself is  $O(p \times N_x \times N_y)$ , which can be significant.
- **Particle colouring / domain decomposition:** partition the domain so that threads always work on non-overlapping cell regions. Requires careful load balancing when particles are non-uniformly distributed.

## 7 Conclusion

- Interpolation scales linearly,  $O(N)$ , confirmed across all configurations and platforms.
- The Lab PC outperforms the HPC cluster for single-threaded workloads due to higher clock frequency, better single-thread cache utilisation, and lower memory latency.
- OpenMP parallelisation of the Mover with four threads yields  $\approx 13.5\times$  super-linear speedup on the Lab PC (cache-size effect) but  $\approx 0.9\times$  on the HPC cluster (overhead and memory bandwidth limitations).

- Parallelising the interpolation step requires additional care to handle write conflicts on shared mesh nodes, with atomic operations or thread-private sub-meshes as the main alternatives.